

Map Search Write Up

Name: **BFS_DFS**

Input: *graph*, a directed graph; *start*, the start node in *graph*; *end*, the end node in the graph; *rac_class*, a restricted access container representing either a queue or a stack depending on whether its a BFS or DFS

Output: Returns *parent*, such that for each node, *node*, in *graph*, $parent_{node}$ = the parent of *node* in the exploration of *graph* starting from *start*

1. **dist** $\leftarrow$ an empty map;
2. **parent** $\leftarrow$ an empty map;
3. for each node, **node**, in **graph** do
 - a. add a new correspondence **node** $\mapsto \infty$ to the map **dist**
 - b. add a new correspondence **node** $\mapsto$ null to the map **parent**
4. **dist**_{start} $\leftarrow$ 0
5. **rac** $\leftarrow$ a new instance of **rac_class**
6. call push(**start**) onto **rac**
7. **while** **rac** is not empty do
 - a. **node** $\leftarrow$ the node returned by calling pop() on **rac**
 - b. if **node** = **end** then
 - i. **return parent**
 - c. for each neighbor, **nbr**, of **node** in **graph** do
 - i. if **dist**_{nbr} = ∞ then
 1. **dist**_{nbr} $\leftarrow$ **dist**_{node} + 1
 2. **parent**_{nbr} $\leftarrow$ **node**
 3. push **nbr** onto **rac**
8. return **parent**

Recursive DFS Cases

Base Case:

When: The start node is equal to the end node, or all the neighbors of the start node have been explored.

What: Return parent and do not recurse.

Recursive Case:

When: The start node is not the end node and has at least one neighbor that has not

been visited yet.

What: Record the start node as the parent of that neighbor for each unexplored neighbor of start node, recursively call dfs using each of those neighbors as the new start node. Continue until every reachable neighbor has been explored, then dfs will eventually reach the base case.

Name: Recursive DFS

Inputs:

- graph: a directed graph representing a street map
- start_node: a node in graph representing the start
- end_node: a node in graph representing the end
- parent: a map that initially has one entry associating the original start_node with null

Output: Modifies the input parent map to associate each visited node with its parent node. Returns parent - a map associating each visited node with its parent node

Steps:

1. if **start_node = end_node** then
 - a. return **parent**
2. else then
 - a. for each neighbor, **nbr**, in the neighbors of **start_node** do
 - i. if **nbr** is not in **parent** then
 1. **parent_{nbr} ← start_node**
 2. **dfs(graph, nbr, end_node, parent)**
3. return **parent**

AStar Search

Inputs:

- graph: a directed graph representing a street map
- start_node: a node in graph representing the start
- end_node: a node in graph representing the end
- edge_distance: a function which takes two nodes and a graph and returns the actual distance between two neighboring nodes
- straight_line_distance: a function which takes two nodes and a graph and returns the straight line distance between two nodes

Outputs: a map associating each visited node with its parent node.

Steps:

1. **openset** $\leftarrow$ an empty map
2. **closedset** $\leftarrow$ an empty set
3. **parents** $\leftarrow$ an empty map
4. **g_value** $\leftarrow$ an empty map
5. add a new correspondence **start_node** $\mapsto$ **straight_line_distance(end_node, start_node, graph)** to **openset**
6. add a new correspondence **start_node** $\mapsto$ **null** to **parents**
7. add a new correspondence **start_node** $\mapsto$ **0** to **g_value**
8. while **openset** is not empty do
 - a. **smallest_fcost** $\leftarrow \infty$
 - b. **smallest_node** $\leftarrow$ null
 - c. for each correspondence **node** $\mapsto$ **f_cost** in **openset** do
 - i. if **f_cost** < **smallest_fcost** then
 1. **smallest_fcost** $\leftarrow$ **f_cost**
 2. **smallest_node** $\leftarrow$ **node**
 - d. if **smallest_node** = **end_node** then
 - i. **return parents**
 - e. add **smallest_node** to **closedset**
 - f. remove the correspondence **smallest_node** $\mapsto$ **smallest_fcost** from **openset**
 - g. for each neighbor, **nbr**, of **smallest_node** in graph do
 - i. if **nbr** is not a key in **openset** and **nbr** is not in **closedset** then

1. add a new correspondence **nbr** $\mapsto$ **smallest_node** to **parents**
2. **g_cost** $\leftarrow$ **g_costs**_{smallest_node} + **edge_distance(smallest_node, nbr, graph)**
3. add a new correspondence **nbr** $\mapsto$ **g_cost** to **g_costs**
4. **f_cost** $\leftarrow$ **g_cost** + **straight_line_distance(end_node, nbr, graph)**
5. add a new correspondence **nbr** $\mapsto$ **f_cost** to **openset**
- ii. else if **nbr** in **openset** then
 1. **g_cost** $\leftarrow$ **g_costs**_{smallest_node} + **edge_distance(smallest_node, nbr, graph)**
 2. if **g_cost**_{nbr} > **g_cost** then
 - a. **f_cost** $\leftarrow$ **g_cost** + **straight_line_distance(end_node, nbr, graph)**
 - b. **openset**_{nbr} $\leftarrow$ **f_cost**
 - c. **parents**_{nbr} $\leftarrow$ **smallest_node**
 - d. **g_costs**_{nbr} $\leftarrow$ **g_cost**

9. return **parents**

Discussion

1. Compare `bfs_dfs` with a `Stack` to recursive `dfs`. How are they the same? How are they different? Which one is better (and why)?

Both `bfs_dfs` with a `stack` and recursive `dfs` use depth first search. This implies they explore their neighbors as deeply as possible before exploring a different node. They are different since `bfs_dfs` with a `stack` uses an explicit stack to track what node to search using LIFO. On the other hand, recursive `dfs` uses an implicit stack made up of python's recursive calls to keep track and decide which node to search. I believe `bfs_dfs` with a `Stack` data structure is better since it returns when the `end_node` is found but recursive `dfs` only ends after backtracking through an entire pathway through a neighbor. The recursive version recursively checks each neighbor as deep as possible whereas the iterative version checks all neighbors before it moves to the next iteration.

2. Which search algorithm yields the best routes? Why do you think it does so?

Astar yields the best routes. I think this is because it considers cost + heuristic, meaning it has a good guess of which node to explore.

3. Which search algorithm yields the worst routes? Why do you think it does so?

Recursive DFS has yields the worst routes. I think this is the case because it does not try to find the shortest path. Instead, it traverses the graph.

4. Why might Google yield better routes than the best algorithm you implemented?

Google might yield better routes because even though astar always find one of the shrotest routes it is not necessarily the best route. Additionally, google can consider many more factors that influence the efficiency of routes such as traffic and weather.